Exercise 05 - Robert Zacchia

Task 01

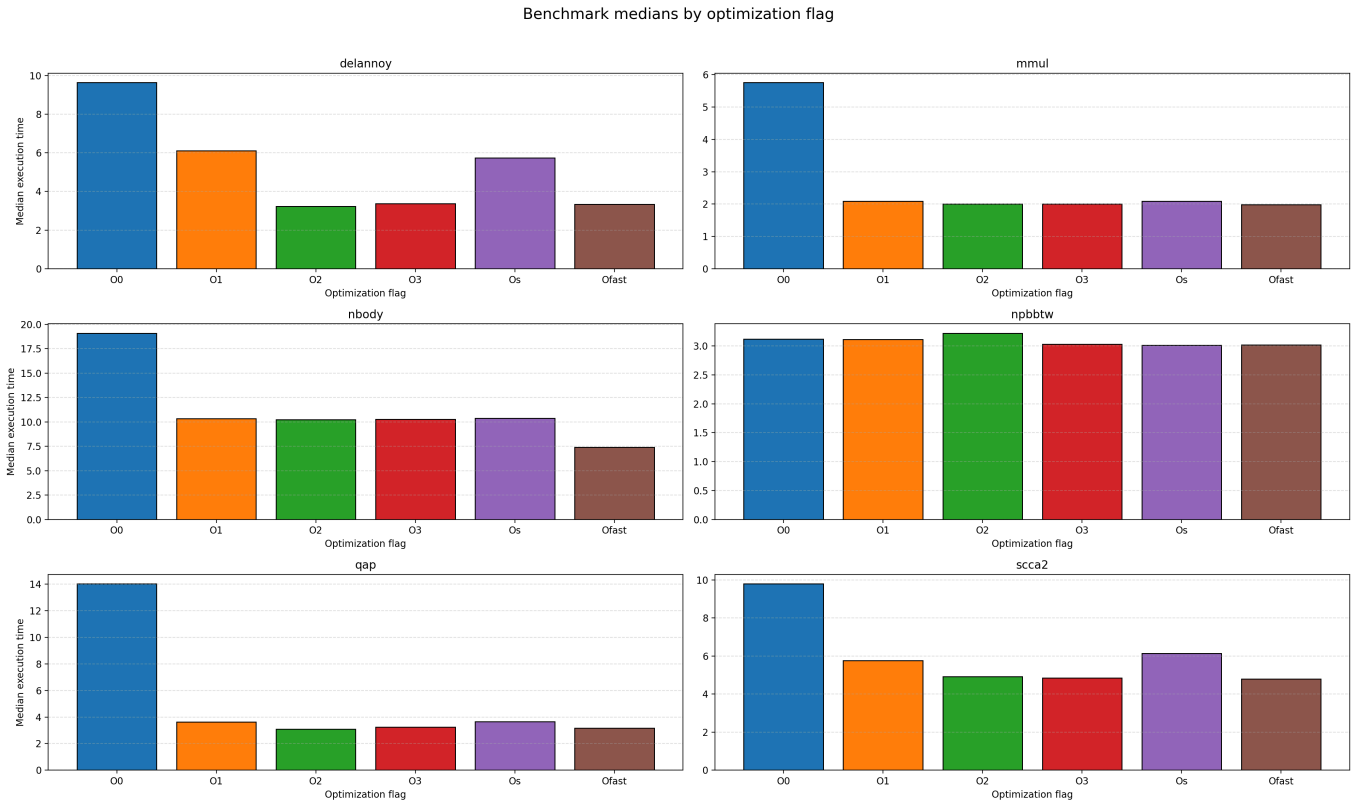

Table 1: Default optimization levels

Program	O0	O1	O2	O3	Ofast	Os
delannoy	9.64 (0.34)	6.11 (0.22)	3.23 (0.28)	3.37 (0.41)	3.33 (0.08)	5.73 (0.21)
mmul	5.76 (0.79)	2.09 (3.41)	2.01 (3.25)	2.01 (3.64)	1.98 (3.15)	2.09 (2.40)
nbody	19.09 (0.20)	10.33 (0.09)	10.23 (0.16)	10.27 (0.15)	7.41 (0.28)	10.38 (0.20)
qap	14.01 (0.25)	3.63 (0.21)	3.10 (0.13)	3.24 (0.35)	3.17 (0.20)	3.66 (0.21)
npbbtw	3.12 (0.35)	3.11 (0.20)	3.22 (0.23)	3.03 (0.41)	3.02 (0.46)	3.01 (0.41)

Discussion

delannoy

- O2 slighlty faster than O3, so cache beheaviour degrades from O2 to O3

mmul

- likely memory bound, since execution time does not decrease after O1

nbody

- same bound as mmul but uses floats instead of integer, thats why Ofast outperforms other optimisations

npb_bt_w

- nearly no impromevements across flags, most likely I/O bound or already optimized code

qap

- large optimization difference from O0 to all others => high compute workload O2 slighty better than O3 (same as dellanoy)

scca2

- O3 best but barely.

Task 02

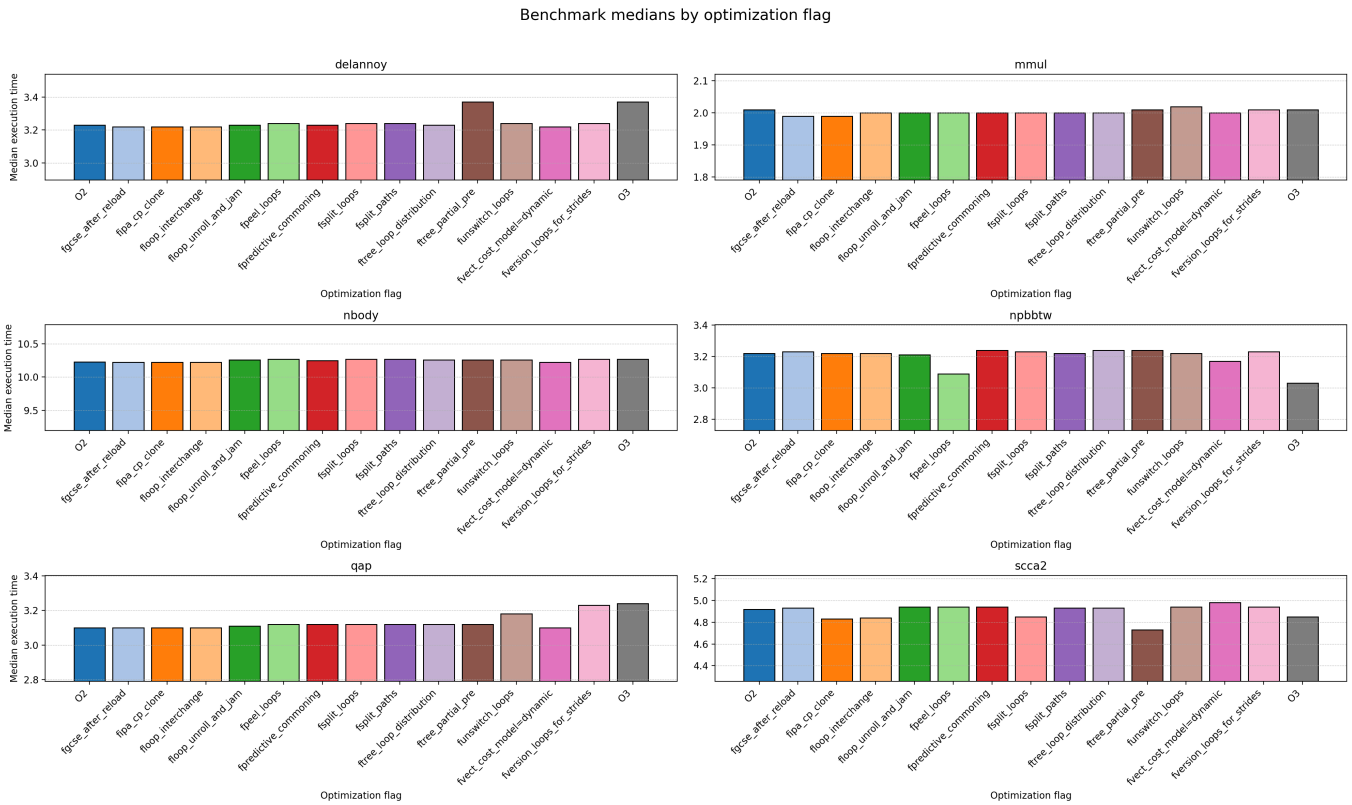

Table 2: O2 individual options and O3

Program	O2	fgcse_after_reload	fipa_cp_clone	floop_interchange	floop_unroll_and_jam	fpeel_loop
delannoy	3.23	3.22	3.22	3.22	3.23	3.24
mmul	2.01	1.99	1.99	2.00	2.00	2.00
nbody	10.23	10.22	10.22	10.22	10.26	10.27
qap	3.10	3.10	3.10	3.10	3.11	3.12

Program	O2	fgcse_after_reload	fipa_cp_clone	floop_interchange	floop_unroll_and_jam	fpeel_loop
npbbtw	3.22	3.23	3.22	3.22	3.21	3.09

Discussion

delannoy

- fvect_cost_model

mmul

- fgcse_after_reload, fipa_cp_clone

nbody

- fgcse_after_reload, fipa_cp_clone, floop_interchange, fvect_cost_model

npb_bt_w

- fpeel_loops

qap

- O2 default is best

scca2

- ftree_partial_pre

Top 3 Flags

- fvect_cost_model=dynamic, only small improvement but across all programs
    - Controls how GCC decides whether to vectorize loops (SIMD).
    - “dynamic” = runtime decision
    - GCC may generate:
        - a vectorized version of the loop
        - a scalar fallback
- fipa_cp_clone, only small improvement but across all programs
    - copies the special cases of a function into its own function to remove branching

```
int foo(int x) {
    if (x == 0) return 1;
    else return x * 2;
}
foo(0);
```

```
    int foo_const_0() {
        return 1;  // branch removed
}
```

- fpeel_loops does only improve npb_bt_w but has the largest single parameter improvement.
    - removes the first or last iterations of a loop if they have conditions checks

    ```
    for (i = 0; i < n; i++) {
    if (i == 0) special_case();
    work();
    }
    ```

    will be transfomed to

    ```
    if (n > 0) {
    special_case();
    work();
    }

    for (i = 1; i < n; i++) {
        work();
    }
    ```

    this way conditions do not get checked in each iterations